

Phase 2: AVL Insertion Imbalances and the Four Rotations

1. Where We Are Now

In Phase 1, you learned how to **detect imbalance** in an AVL tree.

You learned that an AVL tree is a **Binary Search Tree**, also called a BST, with one extra rule: every node must stay balanced.

A BST follows this rule:

left subtree values < node value < right subtree values

An AVL tree also checks the **balance factor** of every node:

Balance Factor = height(left subtree) - height(right subtree)

The valid balance factors are:

-1, 0, +1

The invalid balance factors are:

+2, -2, +3, -3, ...

So Phase 1 taught you this question:

Is the tree balanced or unbalanced?

Phase 2 teaches the next question:

If the tree is unbalanced, how do we fix it?

The answer is:

We fix it using rotations.

2. Main Goal of Phase 2

The main goal of Phase 2 is to understand **rotations after insertion**.

When we insert a new value into an AVL tree, the tree first behaves like a normal BST:

- Smaller value goes left.
- Greater value goes right.

After insertion, we update heights and check balance factors from bottom to top.

If some node gets balance factor `+2` or `-2`, that node is imbalanced.

Then we fix the tree using one of four rotation cases:

LL, RR, LR, RL

3. What Is a Rotation?

A **rotation** is a small rearrangement of tree pointers.

It changes the shape of a small part of the tree so that the tree becomes balanced again.

A rotation does **not** break the BST rule.

That means after rotation, the values are still ordered like this:

left < node < right

Simple meaning:

A rotation fixes the shape of the tree without changing the sorted order of the values.

4. Why Rotations Are Needed

A normal BST can become tall and slow.

Example: inserting values in sorted order:

10, 20, 30

A normal BST becomes:

```
10
 20
  30
```

This is valid as a BST, but it is badly shaped.

The tree is leaning too much to the right.

For AVL trees, this is not allowed because the balance factor of node 10 becomes -2.

So AVL trees use rotations to make the tree short again:

```
  20
 /
10  30
```

Now the tree is balanced.

5. Important Idea: Local Repair

AVL trees do not rebuild the whole tree.

They only repair the small part where the imbalance happened.

Usually, we focus on three important nodes:

1. The imbalanced node
2. Its child on the path of insertion
3. Its grandchild on the path of insertion

The rotation rearranges these three nodes into a balanced shape.

Memory line:

AVL rotation fixes a bad three-node shape by turning it into a balanced triangle.

6. The Four Rotation Cases

There are four possible insertion imbalance cases:

Case	Direction from imbalanced node	Shape type	Rotation type
LL	Left then Left	Straight line	Single rotation
RR	Right then Right	Straight line	Single rotation
LR	Left then Right	Zig-zag	Double rotation
RL	Right then Left	Zig-zag	Double rotation

The case name comes from the path from the **imbalanced node** to the **newly inserted node**.

For example:

```
30 -> left -> left
```

This is an **LL case**.

Another example:

```
30 -> left -> right
```

This is an **LR case**.

7. Straight-Line Cases vs Zig-Zag Cases

There are two main types of shapes.

7.1 Straight-line cases

These are:

```
LL and RR
```

They look like a line.

Example LL:

```

    30
   /
  20
 /
10

```

Example RR:

```

10
  20
    30

```

Straight-line cases use a **single rotation**.

7.2 Zig-zag cases

These are:

LR and RL

They do not form a straight line. They bend like a zig-zag.

Example LR:

```

    30
   /
  10
    20

```

Example RL:

```

10
  30
 /
20

```

Zig-zag cases use a **double rotation**.

A beginner-friendly way to think about LR and RL is:

In a zig-zag case, the middle value moves to the top.

8. LL Case: Left-Left Imbalance

8.1 When LL Happens

LL happens when the new value is inserted into the **left side of the left child**.

Insertion order example:

30, 20, 10

Insert 30 :

30

Insert 20 :

30
/
20

Insert 10 :

30
/
20
/
10

Now node 30 is imbalanced.

Its balance factor is:

BF(30) = +2

The path from 30 to the inserted node 10 is:

left -> left

So this is an **LL case**.

8.2 LL Shape Before Rotation

```
  30
 /
20
 /
10
```

The tree is too heavy on the left side.

8.3 LL Shape After Rotation

The middle node 20 moves up:

```
  20
 /
10  30
```

Now the tree is balanced.

8.4 What Happens During LL Rotation

Before:

```
  p
 /
p1
```

Here:

- `p` is the old root of the imbalanced subtree.
- `p1` is the left child of `p`.

During LL rotation:

- `p1` becomes the new root.
- `p` moves down to the right of `p1`.

For the example:

```
p = 30
p1 = 20
```

After rotation:

```
20 becomes root
30 becomes right child of 20
10 remains left child of 20
```

8.5 LL Rotation Code Idea

```
Node *LLRotation(Node *p) {
    Node *p1 = p->left;

    p->left = p1->right;
    p1->right = p;

    return p1;
}
```

Meaning:

```
p is the old root.
p1 is the left child.
p1 becomes the new root.
p moves down to the right.
```

Important:

The function returns `p1` because `p1` is now the new root of this local subtree.

8.6 Beginner Memory for LL

```
LL = left-left problem
Fix = rotate right
Middle node moves up
```

9. RR Case: Right-Right Imbalance

9.1 When RR Happens

RR happens when the new value is inserted into the **right side of the right child**.

Insertion order example:

10, 20, 30

Insert `10`:

10

Insert `20`:

10

20

Insert `30`:

10

20

30

Now node `10` is imbalanced.

Its balance factor is:

BF(10) = -2

The path from 10 to the inserted node 30 is:

right -> right

So this is an **RR case**.

9.2 RR Shape Before Rotation

```
10
  /
 20
  /
 30
```

The tree is too heavy on the right side.

9.3 RR Shape After Rotation

The middle node 20 moves up:

```
  20
 /
10  30
```

Now the tree is balanced.

9.4 What Happens During RR Rotation

Before:

p

pr

Here:

- p is the old root of the imbalanced subtree.
- pr is the right child of p.

During RR rotation:

- pr becomes the new root.
- p moves down to the left of pr.

For the example:

```
p = 10  
pr = 20
```

After rotation:

```
20 becomes root  
10 becomes left child of 20  
30 remains right child of 20
```

9.5 RR Rotation Code Idea

```
Node *RRRotation(Node *p) {  
    Node *pr = p->right;  
  
    p->right = pr->left;  
    pr->left = p;  
  
    return pr;  
}
```

Meaning:

```
p is the old root.  
pr is the right child.
```

pr becomes the new root.
p moves down to the left.

Important:

The function returns `pr` because `pr` is now the new root of this local subtree.

9.6 Beginner Memory for RR

RR = right-right problem
Fix = rotate left
Middle node moves up

10. LR Case: Left-Right Imbalance

10.1 When LR Happens

LR happens when the new value is inserted into the **right side of the left child**.

Insertion order example:

30, 10, 20

Insert `30`:

30

Insert `10`:

30
/
10

Insert `20`:

30
/
20

```
10
  /
20
```

Now node 30 is imbalanced.

The path from 30 to inserted node 20 is:

left -> right

So this is an **LR case**.

10.2 LR Shape Before Rotation

```
30
 /
10
  \
  20
```

This is a zig-zag shape.

10.3 LR Shape After Rotation

The middle value 20 moves up:

```
20
 / \
10 30
```

Now the tree is balanced.

10.4 What Happens During LR Rotation

In LR, there are three important nodes:

```
p   = old root
pl  = left child
plr = right child of pl
```

For this example:

```
p   = 30
pl  = 10
plr = 20
```

After LR rotation:

```
20 becomes the new root
10 becomes left child of 20
30 becomes right child of 20
```

Beginner idea:

In LR, the middle value climbs to the top.

10.5 LR Rotation Code Idea

```
Node *LRRotation(Node *p) {
    Node *pl = p->left;
    Node *plr = pl->right;

    pl->right = plr->left;
    p->left = plr->right;

    plr->left = pl;
    plr->right = p;

    return plr;
}
```

Meaning:

```
p is the old root.
pl is the left child.
plr is the middle node.
```

```
plr becomes the new root.  
pl becomes the left child.  
p becomes the right child.
```

Important:

The function returns `plr` because `plr` is now the new root of this local subtree.

10.6 Beginner Memory for LR

```
LR = left-right zig-zag  
Fix = middle node moves up  
Result = balanced triangle
```

11. RL Case: Right-Left Imbalance

11.1 When RL Happens

RL happens when the new value is inserted into the **left side of the right child**.

Insertion order example:

```
10, 30, 20
```

Insert `10`:

```
10
```

Insert `30`:

```
10
```

```
30
```

Insert `20`:

10

30
/
20

Now node 10 is imbalanced.

The path from 10 to inserted node 20 is:

right -> left

So this is an **RL case**.

11.2 RL Shape Before Rotation

10

30
/
20

This is also a zig-zag shape.

11.3 RL Shape After Rotation

The middle value 20 moves up:

20
/
10 30

Now the tree is balanced.

11.4 What Happens During RL Rotation

In RL, there are three important nodes:


```
p   = old root
pr  = right child
prl = left child of pr
```

For this example:

```
p   = 10
pr  = 30
prl = 20
```

After RL rotation:

```
20 becomes the new root
10 becomes left child of 20
30 becomes right child of 20
```

Beginner idea:

In RL, the middle value climbs to the top.

11.5 RL Rotation Code Idea

```
Node *RLRotation(Node *p) {
    Node *pr = p->right;
    Node *prl = pr->left;

    pr->left = prl->right;
    p->right = prl->left;

    prl->right = pr;
    prl->left = p;

    return prl;
}
```

Meaning:

```
p is the old root.
pr is the right child.
prl is the middle node.
```

```
prl becomes the new root.  
p becomes the left child.  
pr becomes the right child.
```

Important:

The function returns `prl` because `prl` is now the new root of this local subtree.

11.6 Beginner Memory for RL

```
RL = right-left zig-zag  
Fix = middle node moves up  
Result = balanced triangle
```

12. Big Pattern of All Four Rotations

All four bad shapes become the same balanced triangle.

Example values:

```
10, 20, 30
```

Bad LL shape:

```
  30  
 /  
20  
/  
10
```

Bad RR shape:

```
10  
  
  20  
  
    30
```

Bad LR shape:

```
  30
 /
10
  20
```

Bad RL shape:

```
10
  30
 /
20
```

All of them become:

```
  20
 /
10  30
```

Main memory line:

Bad three-node shape becomes a balanced triangle.

13. The Five Possible BST Shapes with Three Keys

With exactly three keys, there are five possible BST shapes.

Four of them are unbalanced.

One of them is already balanced.

Assume the keys are:

```
10, 20, 30
```

13.1 Shape 1: Left line

30
/
20
/
10

Case:

LL

Needs rotation:

Yes

13.2 Shape 2: Right line

10
20
30

Case:

RR

Needs rotation:

Yes

13.3 Shape 3: Left-right zig-zag

30
/
10

20

Case:

LR

Needs rotation:

Yes

13.4 Shape 4: Right-left zig-zag

10

30
/
20

Case:

RL

Needs rotation:

Yes

13.5 Shape 5: Perfect triangle

```
    20
   /
 10   30
```

Case:

Already balanced

Needs rotation:

No

13.6 Summary Table

Shape	Example	Balanced?	Fix
Left line	30 -> 20 -> 10	No	LL rotation
Right line	10 -> 20 -> 30	No	RR rotation
Left-right zig-zag	30 -> 10 -> 20	No	LR rotation
Right-left zig-zag	10 -> 30 -> 20	No	RL rotation
Perfect triangle	20 root, 10 left, 30 right	Yes	No rotation

14. How to Identify the Correct Rotation

Use this method every time.

Step 1: Insert like a normal BST

First, insert the value using the BST rule:

```
smaller goes left
greater goes right
```

Step 2: Update height from bottom to top

After insertion, update heights from the inserted node back toward the root.

This is because parent height depends on child height.

Step 3: Find the first imbalanced node

Look for the first node whose balance factor becomes:

+2 or -2

That node is the local root of the problem.

Step 4: Trace the path from the imbalanced node to the inserted node

Ask:

From the imbalanced node, did I go left or right?
Then did I go left or right again?

Step 5: Name the case

```
left then left    -> LL
right then right  -> RR
left then right   -> LR
right then left   -> RL
```

Step 6: Apply the correct rotation

```
LL -> single rotation
RR -> single rotation
```

LR -> double rotation
RL -> double rotation

15. Rotation Decision Table

Imbalanced node BF	Child direction	Grandchild direction	Case	Rotation
+2	Left	Left	LL	Single rotation
-2	Right	Right	RR	Single rotation
+2	Left	Right	LR	Double rotation
-2	Right	Left	RL	Double rotation

Another way to remember:

BF +2 means left-heavy.
BF -2 means right-heavy.

Then inspect where the inserted node went next.

16. Single Rotation vs Double Rotation

16.1 Single rotations

Single rotations are used for straight-line problems.

LL and RR

They are called single rotations because one main rotation fixes the problem.

Examples:

LL: rotate right
RR: rotate left

16.2 Double rotations

Double rotations are used for zig-zag problems.

LR and RL

They are called double rotations because the shape needs two directional corrections.

However, as a beginner, you can remember this simpler idea:

In LR and RL, the middle node goes directly to the top.

17. Why Rotations Preserve BST Order

Rotations may look like they change the tree a lot, but they still keep the BST rule.

Example after rotation:

```
  20
 /
10   30
```

This is valid because:

$10 < 20 < 30$

The inorder traversal is still sorted:

10, 20, 30

Important idea:

Rotations change the shape, not the sorted order.

This is why AVL trees remain BSTs after rotations.

18. Pointer Movement in Simple Words

A tree is built using pointers.

Each node has pointers to its children:

```
left
right
```

A rotation changes these child pointers.

For example, in LL rotation:

Before:

```
  30
 /
20
```

After:

```
  20
    30
```

The pointer changes are:

```
20's right pointer points to 30.
30's left pointer changes to whatever 20's old right subtree was.
```

This second part matters more in large trees, because a child may already have a subtree attached.

In Phase 2, the simple three-node examples often do not show those attached subtrees.

Phase 3 studies this more deeply.

19. Why We Return the New Root After Rotation

Every rotation function returns a node pointer.

Example:

```
return pl;
```

or:

```
return pr;
```

or:

```
return plr;
```

or:

```
return prl;
```

This is important because the old root is no longer the root of that local subtree.

Example LL:

Before:

```
30 was root
```

After:

```
20 is root
```

So the rotation must return `20`.

Memory line:

After a rotation, always return the new local root.

20. Height Updates After Rotation

After a rotation, heights may change.

For example, before LL rotation:

```
    30
   /
  20
 /
10
```

The height of 30 was too large because it had a tall left subtree.

After rotation:

```
    20
   /
  10  30
```

Now:

```
height(10) = 1
height(30) = 1
height(20) = 2
```

So in real AVL code, after pointer movement, we must update heights.

Beginner rule:

After rotating, update the lower nodes first, then update the new root.

Phase 2 focuses mainly on shape.

Phase 3 focuses more on full C implementation and height update details.

21. Complete Simple Rotation Demonstration Code

This program demonstrates all four rotations separately.

It is written to show the idea clearly.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
```

```

    int data;
    int height;
    struct Node *left;
    struct Node *right;
} Node;

int max(int a, int b) {
    return (a > b) ? a : b;
}

int height(Node *p) {
    if (p == NULL) {
        return 0;
    }
    return p->height;
}

int bf(Node *p) {
    if (p == NULL) {
        return 0;
    }
    return height(p->left) - height(p->right);
}

Node *newNode(int value) {
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = value;
    p->height = 1;
    p->left = NULL;
    p->right = NULL;
    return p;
}

void update(Node *p) {
    if (p != NULL) {
        p->height = 1 + max(height(p->left), height(p->right));
    }
}

Node *LLRotation(Node *p) {
    Node *pl = p->left;
    Node *plr = pl->right;

    pl->right = p;
    p->left = plr;

    update(p);
    update(pl);
}

```

```

        return pl;
    }

Node *RRRotation(Node *p) {
    Node *pr = p->right;
    Node *prl = pr->left;

    pr->left = p;
    p->right = prl;

    update(p);
    update(pr);

    return pr;
}

Node *LRRotation(Node *p) {
    Node *pl = p->left;
    Node *plr = pl->right;

    pl->right = plr->left;
    p->left = plr->right;

    plr->left = pl;
    plr->right = p;

    update(pl);
    update(p);
    update(plr);

    return plr;
}

Node *RLRotation(Node *p) {
    Node *pr = p->right;
    Node *prl = pr->left;

    pr->left = prl->right;
    p->right = prl->left;

    prl->right = pr;
    prl->left = p;

    update(pr);
    update(p);
    update(prl);
}

```

```

        return prl;
    }

    void preorder(Node *p) {
        if (p == NULL) {
            return;
        }

        printf("%d(h=%d,bf=%d) ", p->data, p->height, bf(p));
        preorder(p->left);
        preorder(p->right);
    }

    int main(void) {
        Node *root;

        root = newNode(30);
        root->left = newNode(20);
        root->left->left = newNode(10);
        update(root->left);
        update(root);
        root = LLRotation(root);
        printf("After LL rotation: ");
        preorder(root);
        printf("\n");

        root = newNode(10);
        root->right = newNode(20);
        root->right->right = newNode(30);
        update(root->right);
        update(root);
        root = RRRotation(root);
        printf("After RR rotation: ");
        preorder(root);
        printf("\n");

        root = newNode(30);
        root->left = newNode(10);
        root->left->right = newNode(20);
        update(root->left);
        update(root);
        root = LRRotation(root);
        printf("After LR rotation: ");
        preorder(root);
        printf("\n");

        root = newNode(10);
        root->right = newNode(30);

```

```

    root->right->left = newNode(20);
    update(root->right);
    update(root);
    root = RLRotation(root);
    printf("After RL rotation: ");
    preorder(root);
    printf("\n");

    return 0;
}

```

22. Program Dry Run in Simple Words

22.1 LL part

The program builds:

```

    30
   /
  20
 /
10

```

Then it calls:

```
LLRotation(root)
```

Result:

```

    20
   /
  10  30

```

22.2 RR part

The program builds:


```
10
  20
    30
```

Then it calls:

```
RRRotation(root)
```

Result:

```
  20
 /
10  30
```

22.3 LR part

The program builds:

```
  30
 /
10
  20
```

Then it calls:

```
LRRotation(root)
```

Result:

```
  20
 /
10  30
```

22.4 RL part

The program builds:

```
10
  30
 /
20
```

Then it calls:

```
RLRotation(root)
```

Result:

```
  20
 /
10  30
```

23. Time and Space Complexity of Rotations

A rotation only changes a fixed number of pointers.

It does not visit every node in the tree.

So a rotation takes:

```
O(1) time
```

It also uses only a few temporary pointer variables.

So extra space is:

```
O(1) space
```

Important:

A rotation is constant time because it always does a fixed amount of work.

Even if the full tree has many nodes, the rotation itself only repairs a small local area.

24. Important Beginner Mistakes

Mistake 1: Choosing LL when the shape is actually LR

Example:

```
  30
 /
10
  20
```

A beginner may think:

```
The imbalance is on the left, so it must be LL.
```

But that is wrong.

You must check two directions:

```
30 -> left -> right
```

So the correct case is:

```
LR
```

Mistake 2: Choosing RR when the shape is actually RL

Example:

```
10
  30
```

```
/
20
```

A beginner may think:

```
The imbalance is on the right, so it must be RR.
```

But the path is:

```
10 -> right -> left
```

So the correct case is:

```
RL
```

Mistake 3: Thinking double rotation is slower in Big O

Double rotation has more pointer changes than single rotation.

But it is still a fixed number of pointer changes.

So it is still:

```
O(1)
```

Fixed work is constant time.

Mistake 4: Forgetting to return the new root

After a rotation, the old root is no longer the local root.

Example LL:

```
Before: 30 is root
After:  20 is root
```

So the rotation must return the new root.

Mistake 5: Thinking rotations change the BST order

Rotations change shape, not order.

After rotation, inorder traversal is still sorted.

Example:

```
  20
 /
10   30
```

Inorder is:

```
10, 20, 30
```

So the BST rule is still correct.

Mistake 6: Forgetting height updates

Pointer changes may change node heights.

After rotating, the program must update heights.

In real code:

```
update old lower nodes first
then update the new root
```

25. Phase 2 Quick Reference Table

Case	Example insertion	Bad shape	Fix	Final local root
LL	30, 20, 10	Left-left line	Single rotation	20
RR	10, 20, 30	Right-right line	Single rotation	20
LR	30, 10, 20	Left-right zig-zag	Double rotation	20

Case	Example insertion	Bad shape	Fix	Final local root
RL	10, 30, 20	Right-left zig-zag	Double rotation	20

26. How to Study Phase 2

Use this simple study order:

1. Memorize what LL, RR, LR, and RL mean.
2. Practice identifying the path from the imbalanced node.
3. Draw the bad shape.
4. Draw the balanced triangle after rotation.
5. Check that the BST order is still correct.
6. Learn the basic pointer movement.
7. Remember that the rotation returns the new root.
8. Remember that rotations are $O(1)$.

27. Practice Questions

Practice 1

Tree:

```

    50
   /
  30
 /
10

```

Path from imbalanced node:

50 -> left -> left

Case:

LL

Fix:

30 becomes the local root.

Result:

```
    30
   /
  10  50
```

Practice 2

Tree:

```
  10
   /
  40
   /
  60
```

Path from imbalanced node:

10 -> right -> right

Case:

RR

Fix:

40 becomes the local root.

Result:

```
    40
   /
  10  60
```

Practice 3

Tree:

```
  50
 /
20
  30
```

Path from imbalanced node:

50 -> left -> right

Case:

LR

Fix:

30 becomes the local root.

Result:

```
  30
 /
20  50
```

Practice 4

Tree:

```
20
  60
 /
40
```


Path from imbalanced node:

20 -> right -> left

Case:

RL

Fix:

40 becomes the local root.

Result:

```
  40
 /
20  60
```

28. Self-Test Questions

Question 1

What does a rotation do?

Answer:

A rotation changes pointers in a small part of the tree to make it balanced again.

Question 2

Does a rotation break the BST rule?

Answer:

No. A rotation keeps the BST order correct.

Question 3

Which cases are straight-line cases?

Answer:

LL and RR

Question 4

Which cases are zig-zag cases?

Answer:

LR and RL

Question 5

What kind of rotation does LL use?

Answer:

Single rotation

Question 6

What kind of rotation does RR use?

Answer:

Single rotation

Question 7

What kind of rotation does LR use?

Answer:

Double rotation

Question 8

What kind of rotation does RL use?

Answer:

Double rotation

Question 9

What is the time complexity of one rotation?

Answer:

$O(1)$

Question 10

Why is a rotation $O(1)$?

Answer:

Because it changes only a fixed number of pointers.

29. One-Sentence Memory Lines

AVL rotations fix imbalance after insertion.

LL and RR are straight-line cases.

LR and RL are zig-zag cases.

Straight-line cases use single rotations.

Zig-zag cases use double rotations.

The middle value becomes the top of the balanced triangle.

Rotations change shape, not sorted order.

Every rotation must return the new local root.

Each rotation takes $O(1)$ time.

30. Final Phase 2 Summary

Phase 2 teaches how AVL trees fix imbalance after insertion.

When a node gets balance factor `+2` or `-2`, the tree is no longer AVL balanced.

To fix it, we find the insertion path from the imbalanced node.

There are four cases:

LL, RR, LR, RL

LL means:

left then left

RR means:

right then right

LR means:

left then right

RL means:

right then left

LL and RR are straight-line cases, so they use single rotations.

LR and RL are zig-zag cases, so they use double rotations.

All four bad shapes are fixed by turning three badly arranged nodes into a balanced triangle.

The final local root is usually the middle value among the three nodes.

Rotations preserve the BST rule.

Rotations also run in constant time:

$O(1)$

After Phase 2, you should be able to:

- Recognize LL, RR, LR, and RL cases.
- Draw the tree before rotation.
- Draw the tree after rotation.
- Explain why the middle node moves up.
- Explain why rotations preserve BST order.
- Understand that rotations are local repairs.
- Understand that each rotation returns the new local root.
- Understand that one rotation is $O(1)$.

If you can do these, you are ready for Phase 3, where rotations are implemented more carefully in C with height updates and larger attached subtrees.